1 Problem Statement

Given a model with N parameters $\theta(t) = \{\theta^i(t)\}_{i=1}^N$ and a loss function $\mathcal{L} : \mathbb{R}^N \to \mathbb{R}$, our goal is to find the most optimal parameter update $\delta\theta$ at each time step t. That is, in gradient descent we have

$$\frac{d\theta^i}{dt} = -g^{ij}\frac{\partial \mathcal{L}}{\partial \theta^j} \implies \delta\theta^i \approx -\eta g^{ij}\frac{\partial \mathcal{L}}{\partial \theta^j}$$

where $\eta \in \mathbb{R}_{>0}$ is the learning rate and g^{ij} is the inverse of the induced metric on the parameter space.

2 Review of Diagonal Metric Update

The original ambient metric in the paper is

$$h = \begin{pmatrix} \gamma & \vec{0} \\ \vec{0}^\top & 1 \end{pmatrix} \in \mathbb{R}^{(N+1)\times(N+1)}$$

where $\gamma \in \mathbb{R}^{N\times N}$. What we really care about is the induced metric on the parameter space, which is given by the pullback of h under the embedding $\phi : \mathbb{R}^N \to \mathbb{R}^{N+1}$ defined by $\phi(\theta^1, \ldots, \theta^N) = (\theta^1, \ldots, \theta^N, f(\theta^1, \ldots, \theta^N))$ where $f : \mathbb{R}^N \to \mathbb{R}$ is some scalar function which we take to be the loss function $\mathcal{L}$. The induced metric g on the parameter space is given by

$$g_{ij} = h_{\alpha\beta}\frac{\partial \phi^\alpha}{\partial \theta^i}\frac{\partial \phi^\beta}{\partial \theta^j} = \gamma_{ij} + \frac{\partial \mathcal{L}}{\partial \theta^i}\frac{\partial \mathcal{L}}{\partial \theta^j}.$$

Then, to calculate the parameter updates $\delta\theta^i$ we need g^{ij}, the inverse of g_{ij}. We can compute this inverse using the Sherman-Morrison-Woodbury formula:

Sherman-Morrison-Woodbury Formula

Let $A \in \mathbb{R}^{n\times n}$ be an invertible matrix, and let $U, V \in \mathbb{R}^{n\times k}$ be matrices. Then, if $C = A + UV^\top$ is invertible, we have

$$C^{-1} = A^{-1} - A^{-1}U(I_k + V^\top A^{-1}U)^{-1}V^\top A^{-1}.$$

This gives us

$$g^{ij} = \gamma^{ij} - \frac{\gamma^{ik}\gamma^{jl}\frac{\partial \mathcal{L}}{\partial \theta^k}\frac{\partial \mathcal{L}}{\partial \theta^l}}{1 + \gamma^{mn}\frac{\partial \mathcal{L}}{\partial \theta^m}\frac{\partial \mathcal{L}}{\partial \theta^n}} = \gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k}$$

where

$$l_i \coloneqq \frac{\partial \mathcal{L}}{\partial \theta^i} \implies l^i = \gamma^{ij} l_j.$$

Thus, the parameter updates are given by

$$\begin{aligned}\delta\theta^i &= -\eta\left(\gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k}\right) l_j \\ &= -\eta l^i \left(1 - \frac{l_j l^j}{1 + l_k l^k}\right) \\ &= -\eta l^i \left(\frac{1}{1 + l_k l^k}\right).\end{aligned}$$

So to summarize, (1) we start with an ambient metric h which induces a metric g on the parameter space, (2) we compute the inverse g^{ij}, and (3) we use g^{ij} to compute the parameter updates $\delta\theta^i$.

3 Off-Diagonal Metric Update

Now, we consider a more general ambient bilinear form of the form

$$h = \begin{pmatrix} \gamma & \vec{a} \\ \vec{b}^\top & 1 \end{pmatrix} \in \mathbb{R}^{(N+1)\times(N+1)}$$

where $\vec{a}, \vec{b} \in \mathbb{R}^N$. Intuitively, the vectors $\vec{a}$ and $\vec{b}$ introduce off-diagonal couplings between the parameter space and the loss function dimension. In general, if $\vec{a} \neq \vec{b}$ this h is not symmetric, so here we simply treat it as a preconditioning matrix rather than a Riemannian metric. The induced bilinear form on the parameter space is then given by

$$\begin{aligned}g_{ij} &= h_{\alpha\beta}\frac{\partial\phi^\alpha}{\partial\theta^i}\frac{\partial\phi^\beta}{\partial\theta^j} \\ &= \gamma_{ij} + a_i\frac{\partial\mathcal{L}}{\partial\theta^j} + b_j\frac{\partial\mathcal{L}}{\partial\theta^i} + \frac{\partial\mathcal{L}}{\partial\theta^i}\frac{\partial\mathcal{L}}{\partial\theta^j} \\ &= \gamma_{ij} + a_i l_j + b_j l_i + l_i l_j.\end{aligned}$$

To compute the inverse g^{ij}, we notice that g can be expressed as a rank-2 update to γ, that is for $U = [a, l], V = [l, b + l] \in \mathbb{R}^{N\times 2}$ we have

$$\begin{aligned}\gamma_{ij} + U_{ik}V_{jk} &= \gamma_{ij} + a_i l_j + (b_j + l_j)l_i \\ &= \gamma_{ij} + a_i l_j + b_j l_i + l_i l_j \\ &= g_{ij}.\end{aligned}$$

Thus, we can apply the Sherman-Morrison-Woodbury formula again to get

$$\begin{aligned}g^{ij} &= \gamma^{ij} - \gamma^{ik}U_{k\alpha}\Big(\delta_{\alpha\beta} + V_{m\alpha}\gamma^{mn}U_{n\beta}\Big)^{-1}V_{l\beta}\gamma^{lj} \\ &= \gamma^{ij} - \gamma^{ik}(a_k, l_k)\begin{pmatrix} 1 + l_m\gamma^{mn}a_n & l_m\gamma^{mn}l_n \\ (b_m + l_m)\gamma^{mn}a_n & 1 + (b_m + l_m)\gamma^{mn}l_n \end{pmatrix}^{-1}\begin{pmatrix} l_n \\ b_n + l_n \end{pmatrix}\gamma^{nj} \\ &= \gamma^{ij} - \gamma^{ik}(a_k, l_k)\begin{pmatrix} 1 + l_n a^n & l_n l^n \\ b_n a^n + l_n a^n & 1 + b_n l^n + l_n l^n \end{pmatrix}^{-1}\begin{pmatrix} l_n \\ b_n + l_n \end{pmatrix}\gamma^{nj}.\end{aligned}$$

For convenience we define the following scalar contractions:

$$c_{al} \coloneqq a_n l^n, \quad c_{ll} \coloneqq l_n l^n, \quad c_{ba} \coloneqq b_n a^n, \quad c_{bl} \coloneqq b_n l^n.$$

Thus, we have

$$\begin{aligned}
g^{ij} &= \gamma^{ij} - \gamma^{ik}(a_k, l_k) \begin{pmatrix} 1 + c_{al} & c_{ll} \\ c_{ba} + c_{al} & 1 + c_{bl} + c_{ll} \end{pmatrix}^{-1} \begin{pmatrix} l_n \\ b_n + l_n \end{pmatrix} \gamma^{nj} \\
&= \gamma^{ij} - \gamma^{ik} \frac{1}{D}(a_k, l_k) \begin{pmatrix} 1 + c_{bl} + c_{ll} & -c_{ll} \\ -(c_{ba} + c_{al}) & 1 + c_{al} \end{pmatrix} \begin{pmatrix} l_n \\ b_n + l_n \end{pmatrix} \gamma^{nj} \\
&= \gamma^{ij} - \gamma^{ik} \frac{1}{D}(a_k, l_k) \begin{pmatrix} (1 + c_{bl} + c_{ll}) l_n - c_{ll}(b_n + l_n) \\ -(c_{ba} + c_{al}) l_n + (1 + c_{al})(b_n + l_n) \end{pmatrix} \gamma^{nj} \\
&= \gamma^{ij} - \gamma^{ik} \frac{1}{D} \left[a_k \big((1 + c_{bl}) l_n - c_{ll} b_n \big) + l_k \big(- (c_{ba} + c_{al}) l_n + (1 + c_{al})(b_n + l_n) \big) \right] \gamma^{nj}
\end{aligned}$$

where

$$\begin{aligned}
D &= \det \begin{pmatrix} 1 + c_{al} & c_{ll} \\ c_{ba} + c_{al} & 1 + c_{bl} + c_{ll} \end{pmatrix} \\
&= (1 + c_{al})(1 + c_{bl} + c_{ll}) - c_{ll}(c_{ba} + c_{al}) \\
&= 1 + c_{al} + c_{bl} + c_{al} c_{bl} + c_{ll} - c_{ll} c_{ba}.
\end{aligned}$$

Quick Sanity Check

If we set $\vec{a} = \vec{0}$ and $\vec{b} = \vec{0}$, we should recover the diagonal metric case. In this case, $U = [0, l]$ and $V = [l, l]$, so

$$c_{al} = 0, \quad c_{ba} = 0, \quad c_{bl} = 0, \quad c_{ll} = l_n l^n,$$

and hence

$$D = 1 + c_{ll}.$$

Then

$$\begin{aligned}
g^{ij} &= \gamma^{ij} - \gamma^{ik} \frac{1}{1 + c_{ll}}(0, l_k) \begin{pmatrix} (1 + 0) l_n - c_{ll} l_n \\ -(0 + 0) l_n + (1 + 0)(0 + l_n) \end{pmatrix} \gamma^{nj} \\
&= \gamma^{ij} - \gamma^{ik} \frac{1}{1 + c_{ll}}(0, l_k) \begin{pmatrix} (1 - c_{ll}) l_n \\ l_n \end{pmatrix} \gamma^{nj} \\
&= \gamma^{ij} - \gamma^{ik} \frac{1}{1 + c_{ll}} l_k l_n \gamma^{nj} \\
&= \gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k},
\end{aligned}$$

which matches our previous result.

Thus, the parameter updates are given by

$$
\begin{aligned}
\delta\theta^i &= -\eta\, g^{ij} l_j \\
&= -\eta \left[\gamma^{ij} - \gamma^{ik} \frac{1}{D} \Big(a_k((1+c_{bl})l_n - c_{ll} b_n) + l_k(-(c_{ba}+c_{al})l_n + (1+c_{al})(b_n+l_n)) \Big) \gamma^{nj} \right] l_j \\
&= -\eta \left[l^i - \frac{1}{D} \gamma^{ik} \Big(a_k((1+c_{bl})l_n - c_{ll} b_n) + l_k(-(c_{ba}+c_{al})l_n + (1+c_{al})(b_n+l_n)) \Big) l^n \right] \\
&= -\eta \left[l^i - \frac{1}{D} \Big(a^i\, c_{ll} + l^i \big(-(c_{ba}+c_{al})c_{ll} + (1+c_{al})(c_{bl}+c_{ll}) \big) \Big) \right] \\
&= -\frac{\eta}{D} \left[D\, l^i - a^i c_{ll} - l^i \big(-(c_{ba}+c_{al})c_{ll} + (1+c_{al})(c_{bl}+c_{ll}) \big) \right] \\
&= -\frac{\eta}{D} \left[l^i \big(1 + c_{al} + c_{bl} + c_{al}c_{bl} + c_{ll} - c_{ll}c_{ba} \big) - a^i c_{ll} - l^i \big(-(c_{ba}+c_{al})c_{ll} + (1+c_{al})(c_{bl}+c_{ll}) \big) \right] \\
&= -\frac{\eta}{D} \left[l^i \big((1+c_{al})(1+c_{bl}) - c_{ll}c_{ba} - 1 \big) - a^i c_{ll} \right] \\
&= -\frac{\eta}{D} \left[l^i (D-1) - a^i c_{ll} \right] \\
&= -\eta \left[\frac{D-1}{D}\, l^i - \frac{c_{ll}}{D}\, a^i \right] \\
&= -\eta \left[\frac{1+c_{al}}{D}\, l^i - \frac{c_{ll}}{D}\, a^i \right].
\end{aligned}
$$

4 Implementation

The off-diagonal optimizers have been implemented in optimisers/jax_offdiag.py and optimisers/torch_offdiag.py.

General Update Direction

The off-diagonal update rules derived above have been implemented in both JAX and PyTorch. In both implementations, the goal is to realise the update

$$\delta\theta^i = -\eta\, g^{ij} r_j,$$

where r_j is the update direction. In ordinary gradient descent one would simply take $r_j = \partial_j \mathcal{L}$, but we can also test other choices such as momentum-based updates $r_j = m_j$ where m_j is an exponentially-smoothed average of past gradients (the usual "momentum" vector). This separation between the geometric direction l_i appearing in the metric and the update direction r_i allows the optimizer to use the metric to shape the descent direction in a more flexible way. Note: the results discussed later will only take $r_j = l_j$.

Update Computational Cost

Recall that the induced bilinear form is

$$g_{ij} = \gamma_{ij} + (a_i l_j + l_i b_j + l_i l_j)\,.$$

Taking $\gamma_{ij} \to \gamma(\mathbb{I}_N)_{ij}$ (that is, we take the γ matrix to be a scalar multiple of the identity matrix so from now on γ is a scalar), adding a hyperparameter ξ to control the strength of the induced correction, and using matrix notation, we have that g is now

$$G = \gamma \mathbb{I}_N + \xi\big(\boldsymbol{a}\boldsymbol{l}^\top + \boldsymbol{l}\boldsymbol{b}^\top + \boldsymbol{l}\boldsymbol{l}^\top\big).$$

We can write this as

$$G = \gamma\, \mathbb{I}_N + \xi\, U V^\top, \qquad U = (\boldsymbol{a},\, \boldsymbol{l}), \qquad V = (\boldsymbol{l},\, \boldsymbol{b} + \boldsymbol{l}).$$

Using the inverse of G we found above, we have

$$G^{-1} r = \frac{1}{\gamma}\boldsymbol{r} - \frac{\xi}{\gamma^2} U\big(\mathbb{I}_2 + V^\top U\big)^{-1} V^\top \boldsymbol{r}.$$

Thus, the contraction $G^{-1}\boldsymbol{r}$ has computational cost $O(N)$ since it only involves the inversion of a 2×2 matrix and scalar products never producing a full $N \times N$ matrix.

Modes for $\boldsymbol{l}$, $\boldsymbol{a}$, and $\boldsymbol{b}$

The optimiser provides several ways to choose the geometric vectors $\boldsymbol{l}$, $\boldsymbol{a}$, and $\boldsymbol{b}$ appearing in the induced off-diagonal bilinear form.

The base direction $\boldsymbol{l}$: The vector $\boldsymbol{l}$ encodes the direction in parameter space along which the loss coordinate of the ambient space stretches. Two choices are provided:

- **Gradient mode**:
$$\boldsymbol{l}_i = \frac{\partial \mathcal{L}}{\partial \theta^i}.$$
This reproduces the behaviour of the original diagonal construction, where the metric is shaped by the instantaneous slope of the loss.

- **Momentum mode**:
$$\boldsymbol{l}_i = \boldsymbol{m}_i,$$
where $\boldsymbol{m}_i$ is the bias-corrected exponential moving average of gradients. In this mode the metric is influenced not only by the current gradient, but by the accumulated direction of travel over recent steps.

Either choice may be paired with $\boldsymbol{r} = \boldsymbol{l}$ or $\boldsymbol{r} = \boldsymbol{m}$ depending on whether the user wishes to precondition the raw gradient or the momentum direction.

The off-diagonal vectors $\boldsymbol{a}$ and $\boldsymbol{b}$: The vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ determine how the parameter coordinates couple to the loss coordinate in the ambient bilinear form. Several built-in modes allow for quick experimentation:

- **Zero mode**: $\boldsymbol{a}_i = 0$ or $\boldsymbol{b}_i = 0$. This eliminates one branch of the off-diagonal coupling.

- **Gradient mode**: $\boldsymbol{a}_i$ or $\boldsymbol{b}_i$ is aligned with $\partial_i \mathcal{L}$, emphasising the local slope of the loss surface.
- **Momentum mode**: $\boldsymbol{a}_i$ or $\boldsymbol{b}_i$ is aligned with the momentum vector $\boldsymbol{m}_i$, incorporating information from recent history.
- **Parameter mode**: $\boldsymbol{a}_i = \theta^i$ or $\boldsymbol{b}_i = \theta^i$, coupling the geometry directly to the position in parameter space.
- **Same-as-a mode**: $\boldsymbol{b}_i = \boldsymbol{a}_i$, producing a symmetric off-diagonal structure.

For full generality, the implementations also accept user-specified choices for $\boldsymbol{a}$ and $\boldsymbol{b}$ or user-provided functions that construct these vectors dynamically at each optimisation step.

Overall, the off-diagonal framework provides a direct and computationally efficient way to explore a richer class of induced metrics. The various modes for $\boldsymbol{l}$, $\boldsymbol{a}$, and $\boldsymbol{b}$ allow the user to interpolate between purely gradient-based behavior, momentum-informed geometry, and more exotic couplings determined by the structure of the model or task.

5 Results

So far I have only tested them on the small examples. The results are in the following commit files:

- HyperbandPruner + RandomSampler
- SuccessiveHalvingPruner + RandomSampler
- NopPruner + RandomSampler

In general the results are not promising as seen in the following figures.

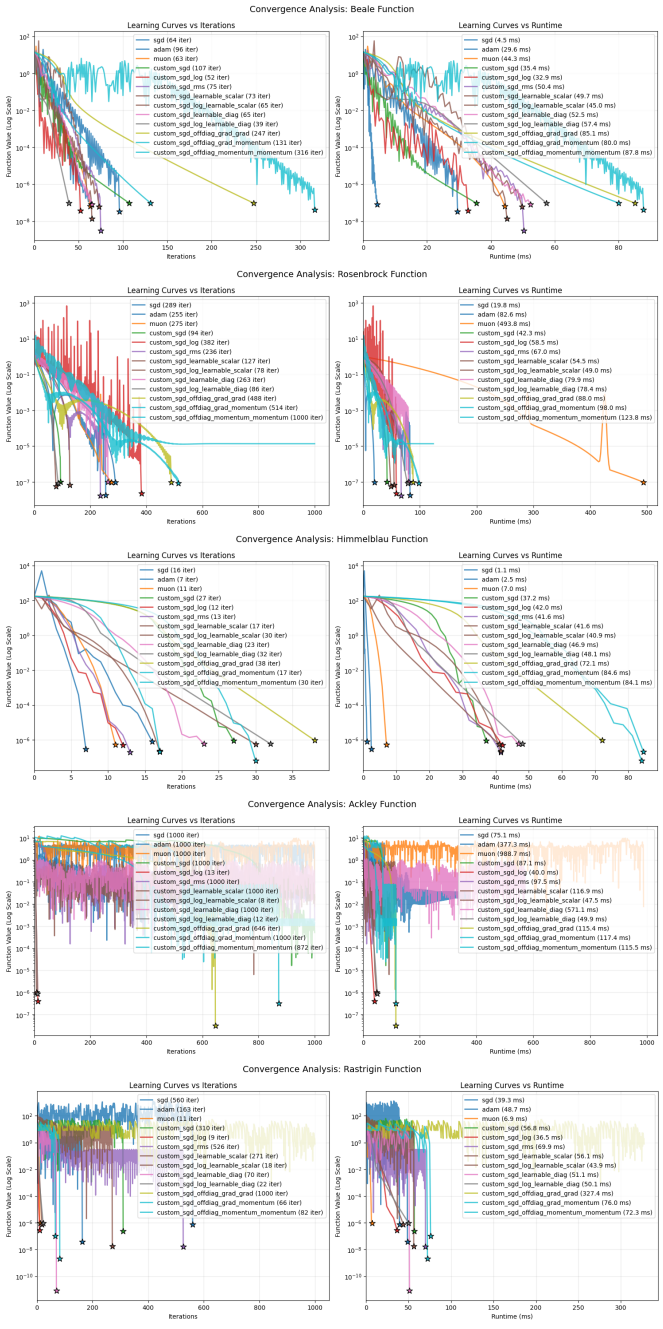

Figure 1: All small example results for NopPruner + RandomSampler, that is no early pruning (so train all models to completion or max 1000 epochs). Each plot shows the best hyper parameter configuration for each optimizer. Note that the naming scheme for the off-diagonal optimizers: `custom_sgd_offdiag_{mode a}_{mode b}`.